

(1)

```
int factorial(int n){
    if(n>0){
        int res=1;
        while(n>0){
            res=res*n;
            n=n-1;
        }
        return res;
    }
    else return 0;
}
```

RAM 程序:

```
                READ 1
                LOAD 1
                JGTZ pos
                JUMP endif
pos:             LOAD=1
                STORE 2
loop1:          LOAD 1
                JGTZ continue
                JUMP print
continue:       MULT 2
                STORE 2
                LOAD 1
                SUB =1
                STORE 1
                JUMP loop1
print:         WRITE 2
endif:         HALT
```

RASP 程序(模拟 RAM, 假定从寄存器 100 开始):

寄存器 编码

100	12
101	1
102	1
103	1
104	16
105	108
106	15
107	134
108	2 //pos
109	2

```

110      2
111      1
112      3
113      2
114      1  //loop1
115      1
116      16
117      120
118      15
119      132
120      8  //continue
121      2
122      3
123      2
124      1
125      1
126      7
127      1
128      3
129      1
130      15
131      114
132      13  //print
133      2
134      18  //endif

```

(2)

```

void sort(){
    int x;
    int a[100];
    int n=0;
    while(cin>>x){
        if(x==0)break;
        else{
            a[n]=x;
            n=n+1;
        }
    }
    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(a[j]>a[i]){
                int temp=a[j];
                a[j]=a[i];

```

```

        a[i]=temp;
    }
}

for(int i=0;i<n;i++){
    cout<<a[i];
}

}

```

RAM 程序:

```

        LOAD =5    //
        STORE 1
input:  READ *1
        LOAD *1
        JZERO sort
        LOAD 1
        ADD =1
        STORE 1
        jump input
sort:   LOAD =5
        STORE 2
loop1:  LOAD 2      //i
        ADD =1
        STORE 3     //j
        LOAD 2
        SUB 1        //i<n
        JZERO print

loop2:  LOAD *3     //a[j]>a[i]
        SUB *2
        JGTZ  swap
        JUMP addJ
swap:   LOAD *2
        STORE 4
        LOAD *3
        STORE *2
        LOAD 4
        STORE *3
addJ:   LOAD 3
        ADD =1
        STORE 3
        SUB 1

```

```

        JZERO addl
        JUMP loop2
addl:   LOAD 2
        ADD =1
        STORE 2
        JUMP LOOP1
print:  LOAD =5
        STORE 2
loops:  LOAD 2
        SUB 1
        JZERO endif
        LOAD 2
        WRITE *2
        ADD =1
        STORE 2
        JUMP loops
endif:  HALT

```

RASP 程序：对应 RASP 指令编码将 RAM 程序的指令与操作数对应存入存储器中即可，但当用 RASP 模拟 RAM 的间接寻址时，假定借助于寄存器 x 执行指令 YYY *i，下面为 RASP 模拟 RAM 的一条间接寻址指令

寄存器 内容

100	YYY 指令编码
101	x
102	1
103	r+i
104	5
105	r
106	3
107	111
108	1
109	x
110	6
111	-

参考课本 p229

表 8-7 用 RASP 指令模拟 RAM 指令 SUB * i

寄存器	内 容	指 令
100	3	STORE 1
101	1	
102	1	LOAD $r+i$
103	$r+i$	
104	5	ADD = r
105	r	
106	3	STORE 111
107	111	
108	1	LOAD 1
109	1	
110	6	SUB b
111	—	(其中 b 是 RAM 寄存器 i 中内容 $c(i)$ 加上 r)